

Smart Contract Audit Report

Contract: sample_contract

Date: 2025-07-08 23:07:04

Risk Summary

Risk Score: 30/100

Risk Level: Medium

Vulnerabilities Found: 3

Vulnerability Findings

1. Unchecked arithmetic operation

Severity: Unknown

Location: balance += amount

Description: Operation 'balance += amount' might cause overflow/underflow. Consider using safe math checks.

2. Unchecked arithmetic operation

Severity: Unknown

Location: balance -= amount

Description: Operation 'balance -= amount' might cause overflow/underflow. Consider using safe math checks.

3. Unchecked arithmetic operation

Severity: Unknown

Location: balance -= amount

Description: Operation 'balance -= amount' might cause overflow/underflow. Consider using safe math checks.

AI-Powered Analysis

Audit Summary for sample_contract

Vulnerability 1: Unchecked Arithmetic Operation

Severity: Medium

Impact: Potential overflow or underflow of the `balance` variable, leading to unexpected behavior or unauthorized access to funds.

*** **Potential Exploit Paths:****

- + An attacker could manipulate the `amount` variable to cause an overflow or underflow, potentially allowing them to drain the contract's funds or gain unauthorized access.
- + In a multi-user scenario, an attacker could exploit the vulnerability to steal funds from other users.

*** **Mitigation Strategies:****

- + Use safe math libraries, such as OpenZeppelin's SafeMath, to perform arithmetic operations. These libraries include built-in checks for overflow and underflow.
- + Implement explicit checks for overflow and underflow conditions before performing arithmetic operations.
- + Consider using uint256 variables instead of uint, as they provide a larger range and reduce the likelihood of overflow.

****Advice for Developers:****

When performing arithmetic operations, always assume that the input values can be maliciously crafted. Use safe math libraries or implement explicit checks to prevent overflow and underflow. Remember that uint variables have a limited range, and using uint256 can provide an additional layer of security.

****Vulnerability 2 and 3: Unchecked Arithmetic Operation (identical vulnerabilities)****

*** **Severity:** Medium**

*** **Impact:** Same as Vulnerability 1.**

*** **Potential Exploit Paths:** Same as Vulnerability 1.**

*** **Mitigation Strategies:** Same as Vulnerability 1.**

****Additional Advice:****

- * It's essential to address these vulnerabilities, as they can have severe consequences for the contract's security and integrity.
- * Consider implementing a comprehensive testing strategy to identify and prevent similar vulnerabilities in the future.
- * Regularly review and update your contract's code to ensure it remains secure and compliant with best practices.

By addressing these vulnerabilities and implementing the recommended mitigation strategies, you can significantly improve the security and reliability of the `sample\_contract` contract.

Generated by DeFi Sentinel